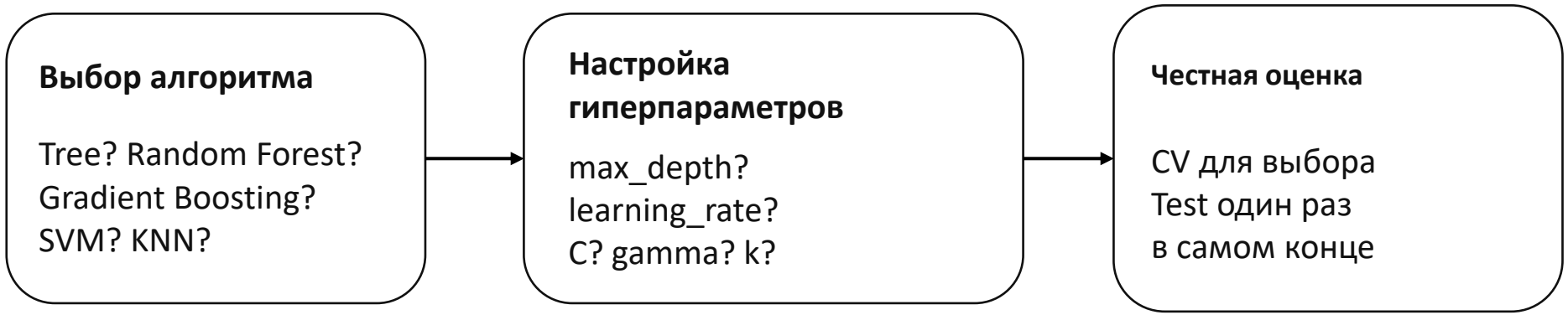
Оптимизация гиперпараметров

Grid Search / Random Search / Cross-validation



Что мы вообще оптимизируем?

Сначала выбираем подход, потом настраиваем его под данные.



Нет универсально лучшего алгоритма

Качество всегда зависит от конкретных данных и выбранной метрики.

Baseline

Простое решение,
с которым сравниваемся.

SOTA

Лучшие известные
результаты для задачи.

No Free Lunch

Если алгоритм силен
на одних данных, это не
делает его лучшим везде.

Практика

Сравниваем несколько
разумных моделей
одинаково и честно.



Параметры и гиперпараметры - не одно и то же

Параметры модель учит. Гиперпараметры задаем мы до обучения.

Параметры модели

Находятся внутри fit()

Примеры:
- веса линейной модели
- пороги и значения в листьях дерева

Гиперпараметры

Задаются до fit()

Примеры:
- max_depth
- n_estimators
- learning_rate
- C, gamma, k

Задача настройки гиперпараметров

Ищем значения, которые дают лучшее качество на кросс-валидации.

$$\lambda^* = \text{argmax} \quad CV_score(model(\lambda))$$

λ – набор гипепараметров

Argmax - выбираем вариант с лучшим значением метрики

CV_score - качество на кросс-валидации

Train / CV / Test: у каждого своя роль

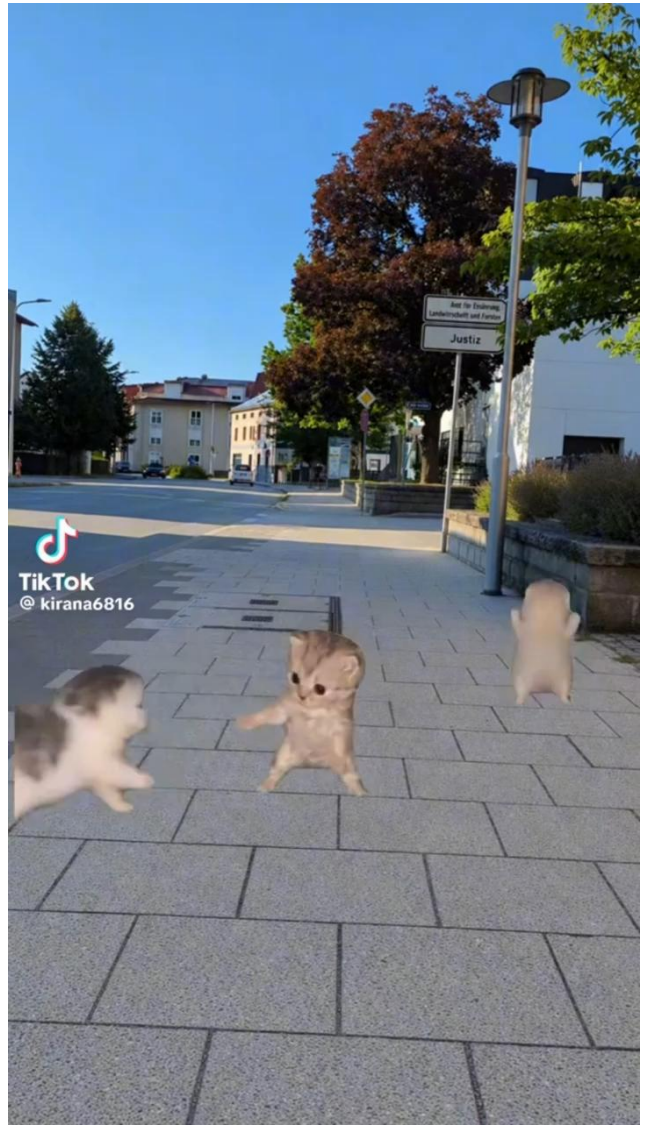
Гиперпараметры выбираем не по test.



Подбор гиперпараметров - это часть обучения. Поэтому test должен оставаться "неизвестным" до самого конца.

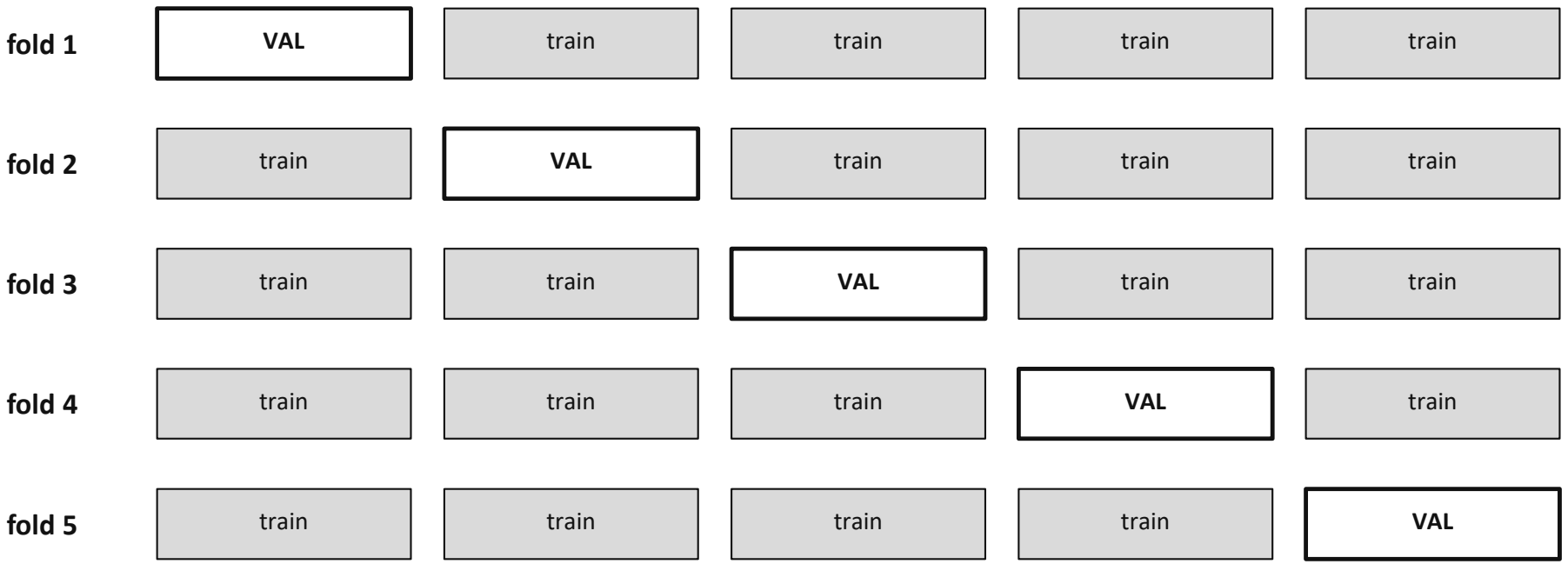
Почему нельзя подбирать параметры по test?

1. Попробовали max_depth=3 -> test F1 = 0.74
2. Попробовали max_depth=5 -> test F1 = 0.76
3. Попробовали еще 30 вариантов и выбрали лучший test



Кросс-валидация

Делим train на несколько частей и по очереди используем каждую как validation.



Для классификации обычно используем StratifiedKFold, чтобы доля классов в фолдах была похожей.

Каждый объект побывал в validation ровно один раз.

Итог = среднее качество по фолдам.

Среднее качество - еще не все

Смотрите не только mean CV score, но и разброс по фолдам.

Модель A

F1 = 0.78

Фолды: 0.77 · 0.79 · 0.78 · 0.78 · 0.78

Одинаковое среднее

Модель B

F1 = 0.78

Фолды: 0.66 · 0.89 · 0.72 · 0.86 · 0.77

Но вторая модель гораздо менее стабильна

Типы гиперпараметров

Пространство поиска состоит не только из списков чисел.

Категориальные

kernel = {linear, rbf}
criterion = {gini, entropy}

Целочисленные

max_depth = 2...20
n_neighbors = 3...31

Непрерывные

learning_rate
C · gamma · alpha

Условные

degree имеет смысл
только при kernel="poly"

Перед поиском сначала определяем тип каждого гиперпараметра и разумный диапазон.

Линейная или логарифмическая шкала?

Параметры регуляризации часто меняют на порядки, а не на одинаковый шаг.

Плохо для C

0.1 0.2 0.3 0.4 0.5

Мы исследуем только очень узкий диапазон.

Лучше

$$10^{-4} \quad 10^{-2} \quad 1 \quad 10^{2} \quad 10^{4}$$

Проверяем несколько порядков величины.

Если параметр может отличаться в 1000 раз, шаг +1 почти бессмыслен. Для Random Search удобно задавать log-uniform распределение.

Не делайте пространство поиска бессмысленно большим

Больше комбинаций не означает лучше эксперимент.

НЕ НАДО

- max_depth = [1, 2, 3, ..., 1000]
- learning_rate = [0.00001, ..., 1] с шагом 0.00001
- 20 параметров "на всякий случай"



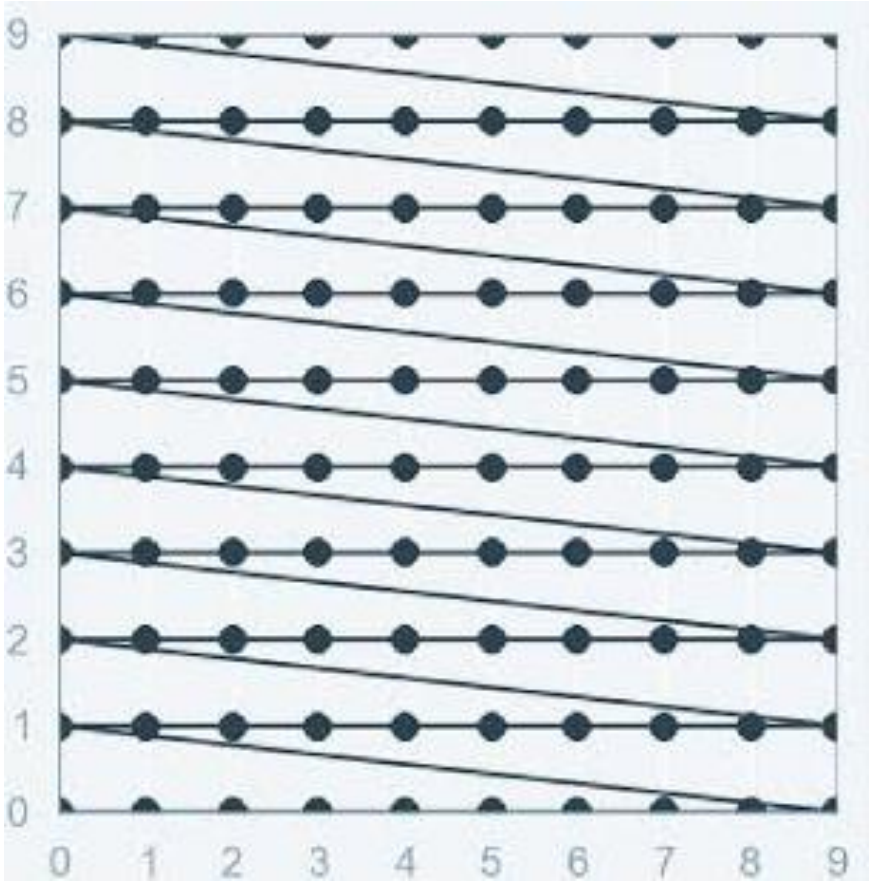
ЛУЧШЕ

- 3-5 значимых параметров
- разумные диапазоны из смысла модели
- грубый поиск -> затем уточнение области



Grid Search

Заранее задаем сетку и проверяем каждую комбинацию.



Проверяем ВСЕ точки сетки

Плюсы
- просто
- воспроизводимо
- понятно, что проверили

Минусы
- быстро растет число комбинаций
- плох при многих параметрах

Хорош для небольшого пространства поиска.

GridSearchCV: минимальный рабочий код

Параметры Pipeline задаются через двойное подчеркивание.

```python
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import GridSearchCV
from sklearn.pipeline import Pipeline

pipe = Pipeline([
    ("model", RandomForestClassifier(random_state=42))
])

param_grid = {
    "model__n_estimators": [100, 300, 500],
    "model__max_depth": [None, 5, 10],
    "model__min_samples_leaf": [1, 2, 4],
}

grid = GridSearchCV(
    pipe, param_grid, scoring="f1", cv=5, n_jobs=-1
)
grid.fit(X_train, y_train)

```

param_grid	какие значения проверить
scoring="f1"	что считаем качеством
cv=5	5 фолдов
n_jobs=-1	все ядра CPU
fit()	запускает весь перебор

Сколько моделей реально обучает Grid Search?

Количество комбинаций умножается на число фолдов.

3 значения n_estimators × **3 значения max_depth** × **3 min_samples_leaf** = **27 комбинаций**

27 комбинаций × 5 фолдов = 135 обучений модели

И это еще маленькая сетка!!!! 

Поэтому перед Grid Search всегда оцениваем вычислительный бюджет.

Что достать из GridSearchCV

Нам нужен не только best_params_.

```python
print(grid.best_params_)
print(grid.best_score_)

best_model = grid.best_estimator_
results = pd.DataFrame(grid.cv_results_)

results[[
    "params",
    "mean_test_score",
    "std_test_score",
    "mean_fit_time",
]].sort_values("mean_test_score", ascending=False)

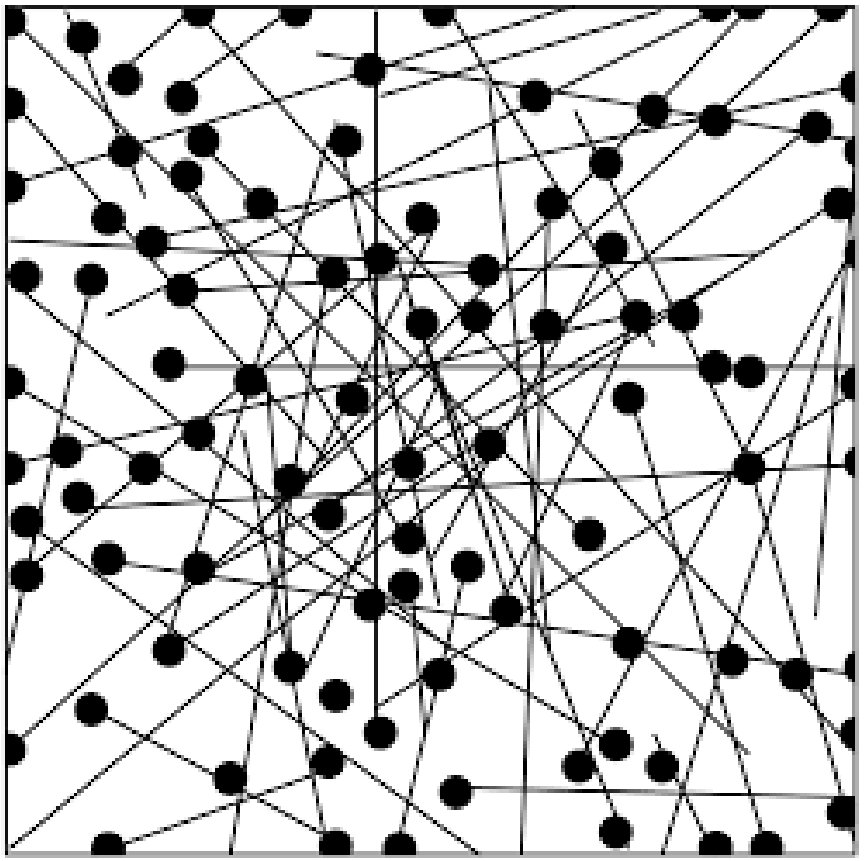
```

best_params_	лучшие значения
best_score_	средний CV score
best_estimator_	готовая refit-модель
cv_results_	все комбинации, время, mean/std

ЛР3: сохраняйте и анализируйте таблицу результатов, а не только одну лучшую строку.

Random Search

Не строим полную сетку. Сэмплируем ограниченное число комбинаций.



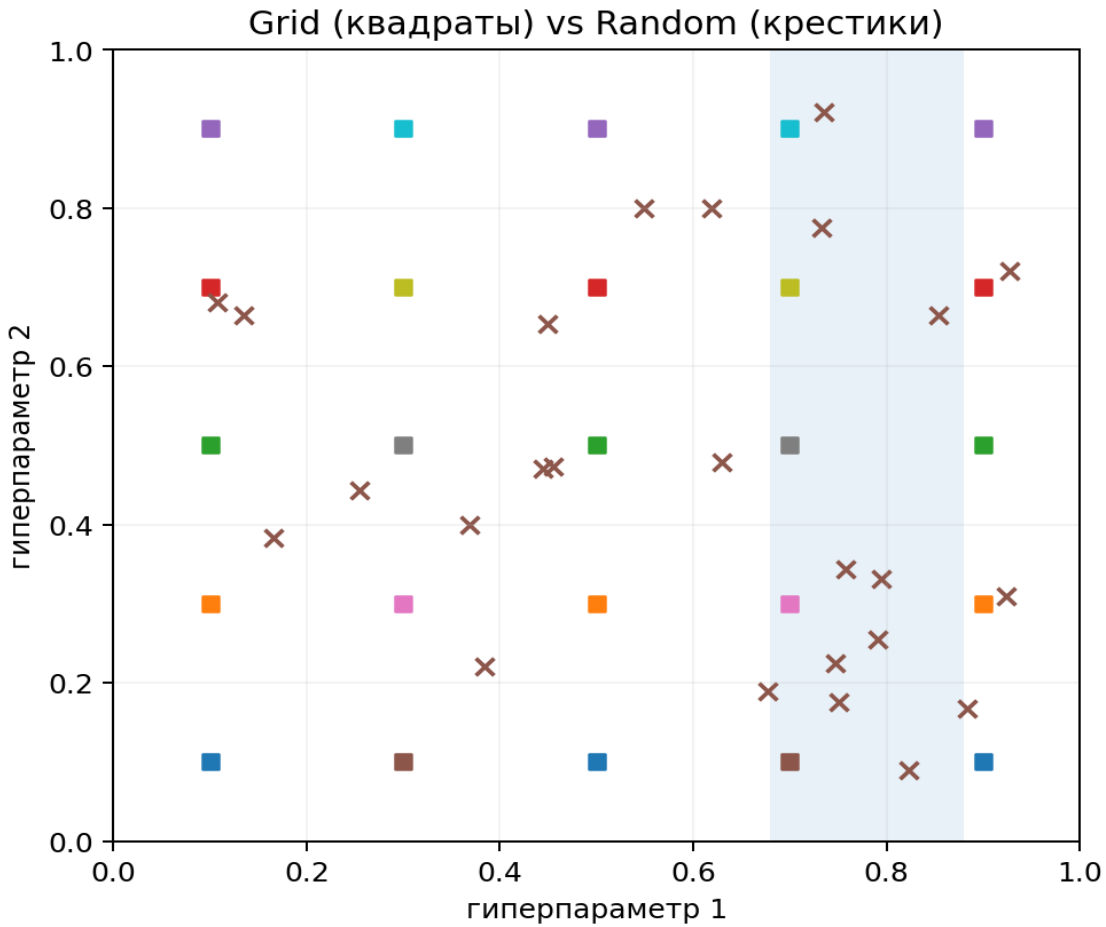
n_iter = 30

Проверим ровно 30 случайных комбинаций, даже если пространство огромное.

Плюс	лучше покрывает большие пространства при фиксированном бюджете
Минус	результат зависит от random_state и n_iter

Почему Random Search часто эффективнее

Если реально важен только один параметр, сетка тратит много попыток зря.



Grid часто повторяет одинаковые значения важного параметра, меняя менее важный.

Random при том же количестве запусков проверяет больше разных значений по каждой оси.

Особенно полезно, когда параметров много и мы не знаем заранее, какие из них важнее.

RandomizedSearchCV: минимальный код

Числовые гиперпараметры можно задавать распределениями.

```python
from scipy.stats import randint
from sklearn.model_selection import RandomizedSearchCV

param_dist = {
    "model__n_estimators": randint(100, 600),
    "model__max_depth": [None, 4, 6, 8, 12],
    "model__min_samples_leaf": randint(1, 8),
}

random_search = RandomizedSearchCV(
    estimator=pipe,
    param_distributions=param_dist,
    n_iter=30,
    scoring="f1",
    cv=5,
    random_state=42,
    n_jobs=-1,
)

random_search.fit(X_train, y_train)

```

randint(100, 600) случайные целые значения

n_iter=30 30 комбинаций

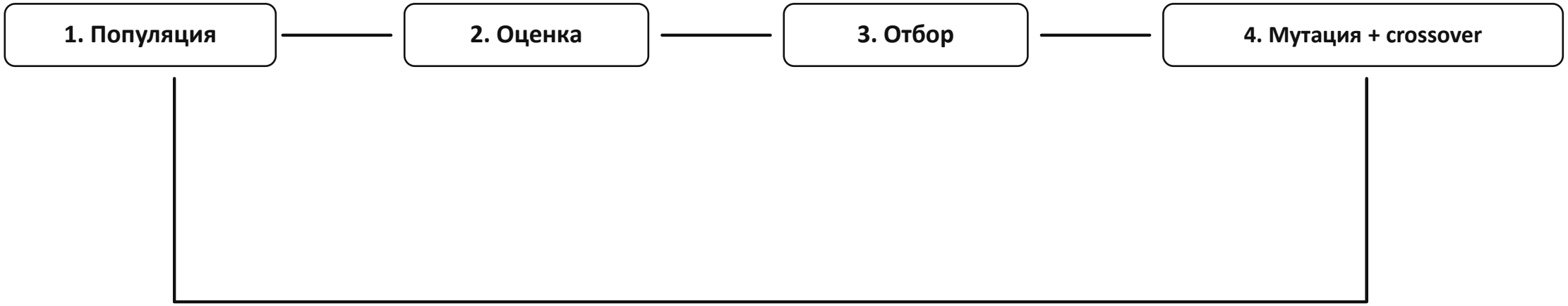
random_state=42 фиксируем случайность

Бюджет задаем явно через n_iter.

Эволюционные алгоритмы

Еще один способ искать хорошие настройки в функции-"черном ящике".

Каждая "особь" = набор гиперпараметров.

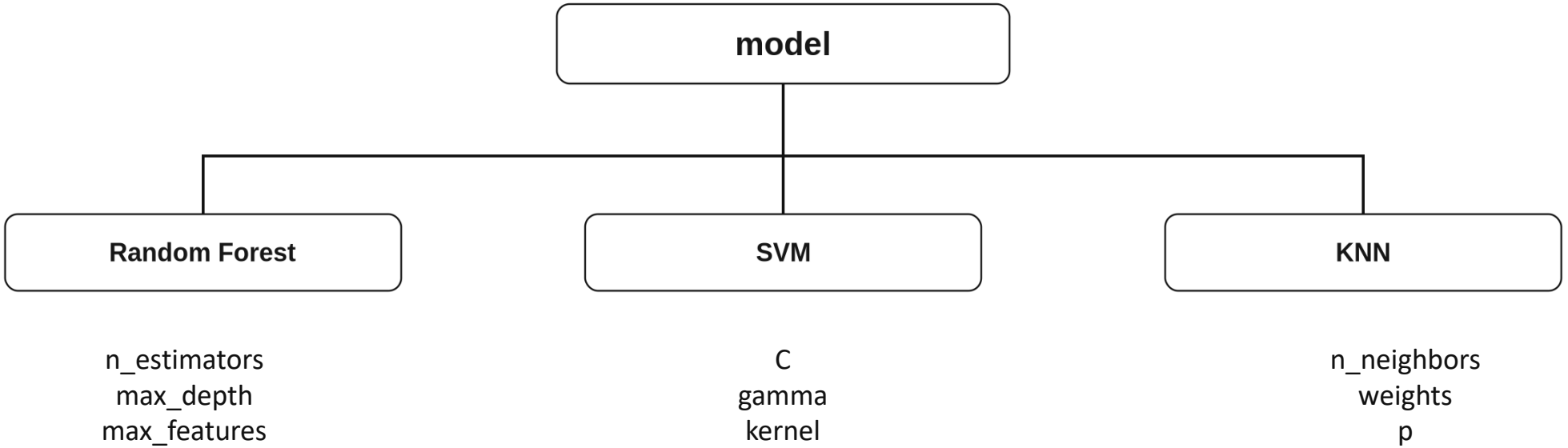


Оставляем более успешные наборы, создаем новые комбинации и повторяем несколько поколений.

Плюс: не нужен градиент и точная формула функции.
Минус: может быть дорого по числу обучений.

Выбор алгоритма и гиперпараметров можно объединить

Тогда сам тип модели становится еще одним гиперпараметром.



Получается древовидное пространство: разные ветки имеют разные гиперпараметры.

Что такое синтетический датасет?

Это данные, которые мы не собрали в реальном мире, а сгенерировали по заданным правилам.

Реальный датасет

Пассажиры Titanic
Вино из лаборатории
Медицинские пациенты

vs

Синтетический датасет

Объекты создаёт генератор
Мы сами задаём правила
И сами знаем сложность

Синтетика не "лучше" реальных данных.
Это учебный стенд: меняем одно условие и смотрим, что произошло с моделью.

Зачем синтетика именно в теме гиперпараметров?

Она позволяет менять сложность данных отдельно от сложности модели.

1	**Контролируем данные**	можем отдельно добавить шум, дисбаланс или лишние признаки
2	**Проверяем одну гипотезу**	например, что произойдёт с max_depth при росте шума
3	**Знаем "правду" генератора**	сколько признаков реально информативны и сколько создано как шум
4	**Повторяем эксперимент**	одинаковый random_state -> одинаковый набор у всех студентов

В реальном датасете мы часто не знаем, почему модель стала хуже. В синтетике причину можно создать своими руками.

Как сгенерировать синтетическую классификацию

make_classification создаёт X и y с контролируемыми свойствами.

```python
from sklearn.datasets import make_classification

X, y = make_classification(
    n_samples=1000,
    n_features=12,
    n_informative=6,
    n_redundant=2,
    weights=[0.65, 0.35],
    class_sep=1.0,
    flip_y=0.05,
    random_state=42,
)

```

Что получаем?

X матрица признаков: 1000 × 12

y готовые метки классов 0 / 1

Важно никаких настоящих пассажиров или пациентов тут нет. Это математически созданные объекты.

random_state=42 делает генерацию воспроизводимой

make_classification: разберём параметры по смыслу

Это не параметры модели. Это настройки самого генератора данных.

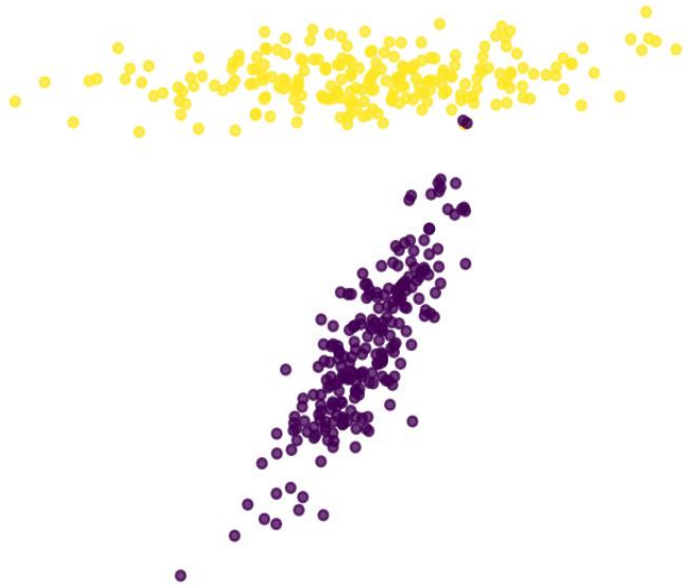
n_samples=1000	создать 1000 объектов
n_features=12	у каждого объекта будет 12 признаков
n_informative=6	6 признаков реально несут информацию о классе
n_redundant=2	2 признака получены из информативных и частично дублируют сигнал
weights=[0.65, 0.35]	примерно 65% объектов класса 0 и 35% класса 1
class_sep=1.0	насколько далеко классы расположены друг от друга
flip_y=0.05	примерно у 5% объектов случайно испортить метку класса
random_state=42	получать один и тот же датасет при повторном запуске

Главное: мы можем менять ОДИН параметр генератора и видеть, как на одном контролируемом изменении ведут себя гиперпараметры модели.

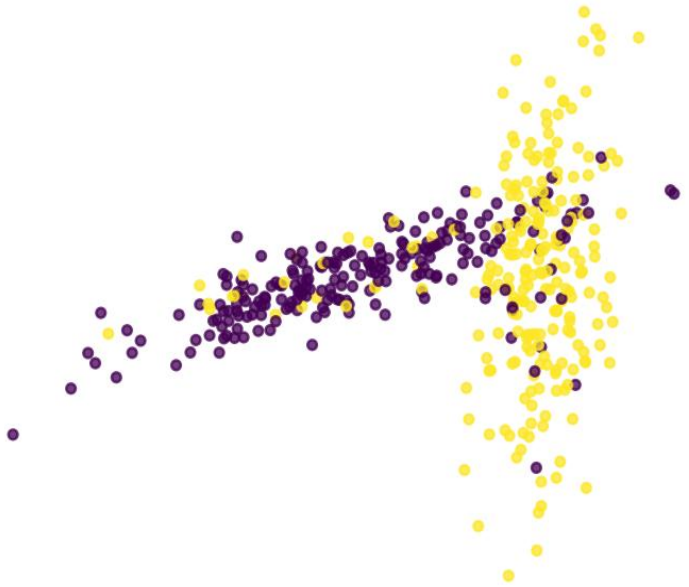
Одни и те же 2 признака — три разные задачи

Меняем параметры генератора, а не модель.

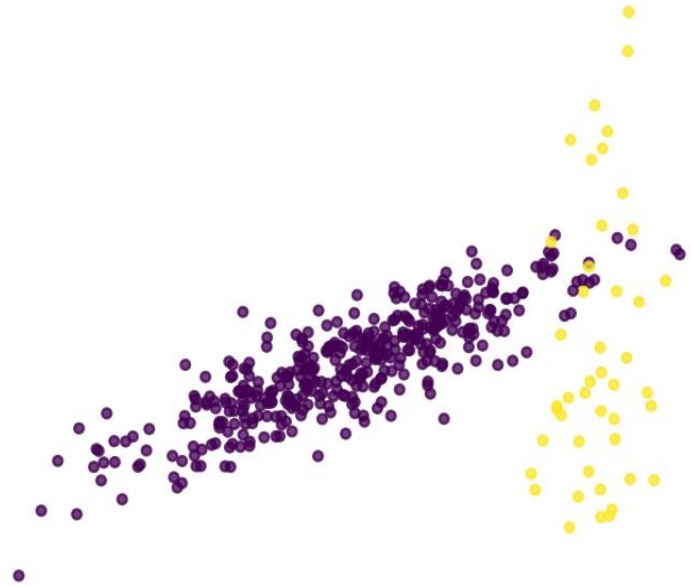
Легко: class_sep=2.0



Шум: flip_y=0.20



Дисбаланс: 90 / 10



class_sep ↑ классы легче разделить

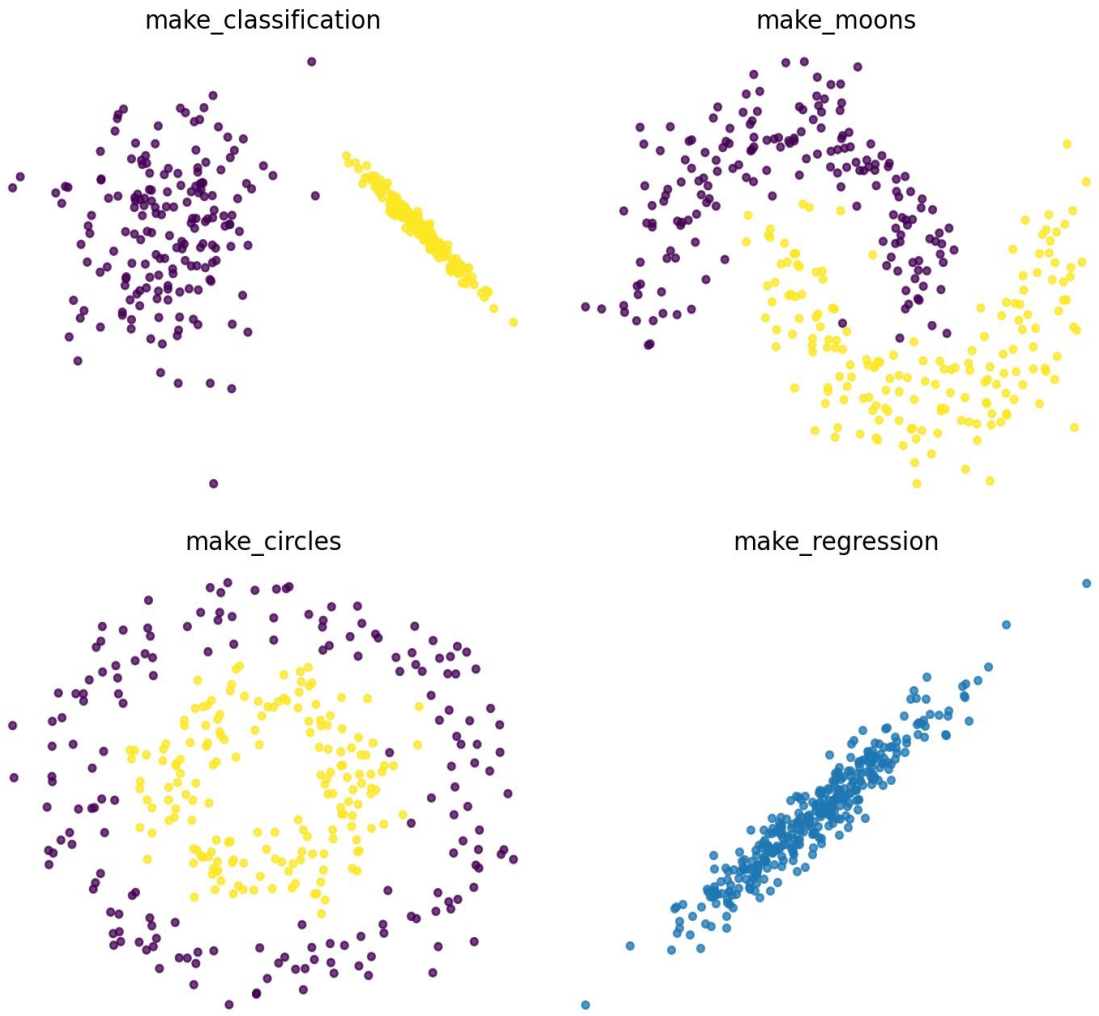
flip_y ↑ больше случайных ошибок в y

weights создают дисбаланс классов

Так можно отдельно проверить: что станет с лучшим max_depth, C или k, если задача стала шумнее или несбалансированнее.

Синтетика бывает разной

Выбираем генератор под тот эффект, который хотим показать.



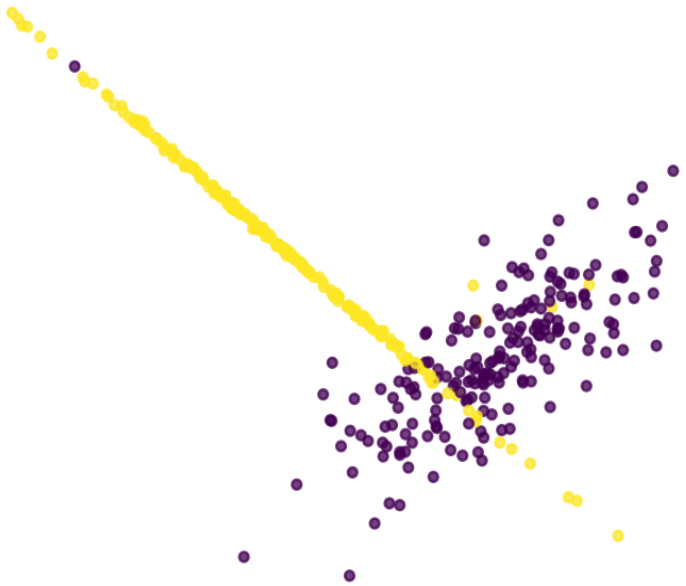
make_classification	обычная классификация: шум, лишние признаки, дисбаланс
make_moons	нелинейная граница: хорошо видно, что линейной модели может не хватить
make_circles	классы вложены друг в друга: удобный тест для kernel и локальных методов
make_regression	регрессия: можно управлять числом признаков и уровнем шума

То есть синтетический датасет выбирают не "какой красивее", а под конкретный вопрос к алгоритму.

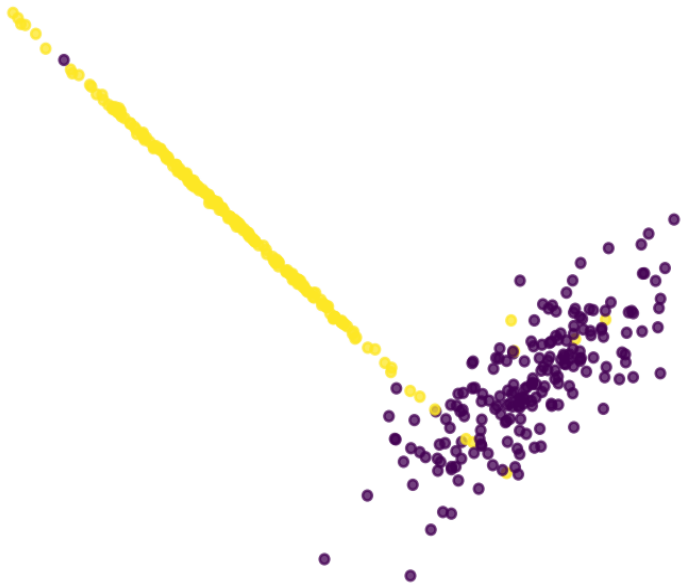
Пример контролируемого эксперимента

Меняем только class_sep и смотрим, как меняется подбор модели.

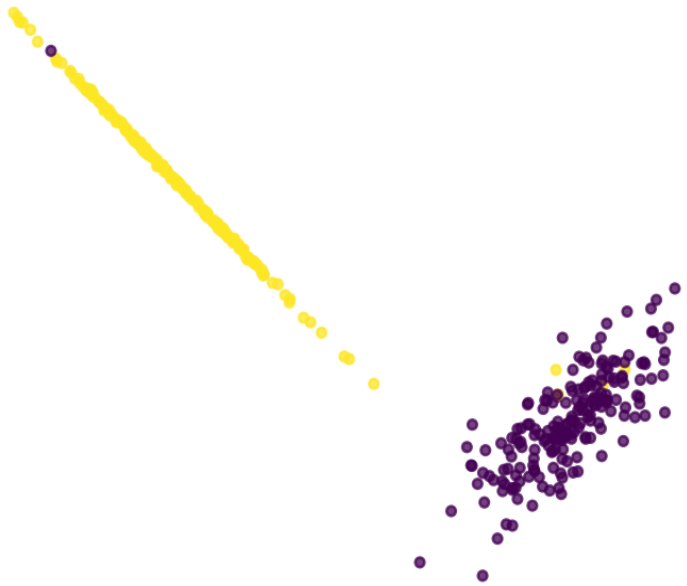
class_sep = 0.6



class_sep = 1.0



class_sep = 1.8



сгенерировали 3 версии данных

для каждой подбираем один и тот же max_depth

сравниваем best CV и выбранную глубину

Так мы можем говорить именно о влиянии сложности данных, потому что всё остальное оставили одинаковым.

Что синтетика может показать — и чего не может

Это инструмент для эксперимента, а не замена реальным данным.

МОЖЕТ

- показать underfit / overfit
- проверить чувствительность к шуму
- проверить влияние дисбаланса
- сравнить методы при одинаковых условиях
- быстро отладить код поиска

НЕ МОЖЕТ

- доказать качество на реальных пользователях
- воспроизвести все зависимости реального мира
- заменить предметный анализ
- гарантировать, что лучшие настройки перенесутся на другой датасет

Настоящие задачи: гиперпараметры зависят от контекста

Один и тот же метод поиска решает очень разные прикладные задачи.

Titanic

Классификация

Метрика F1

Что настраиваем

SVM: C, gamma
RF: depth, n_estimators

Wine Quality

Регрессия

Метрика RMSE

Что настраиваем

GB: learning_rate, depth
RF: depth,
min_samples_leaf

Мед. скрининг

Классификация

Метрика Recall / F1

Что настраиваем

class_weight, C, threshold

Антифрод

Редкий класс

Метрика PR-AUC / Recall

Что настраиваем

class_weight, depth,
min_samples_leaf

Реальный пример: Titanic + SVM

Из ЛР2 уже есть задача и метрика. В ЛР3 меняем только настройки модели.

```python
param_grid = {
    "model__C": [0.1, 1, 10, 100],
    "model__gamma": ["scale", 0.01, 0.1, 1],
}

grid = GridSearchCV(
    estimator=svm_pipeline,
    param_grid=param_grid,
    scoring="f1",
    cv=5,
    n_jobs=-1,
)

grid.fit(X_train, y_train)

```

C	насколько сильно штрафуем ошибки train
gamma	насколько локальной может быть RBF-граница
scoring="f1"	потому что в ЛР2 именно F1 — основная метрика

Реальный пример: Wine Quality + Gradient Boosting

Для регрессии логика та же, но оптимизируем RMSE.

```python
param_grid = {
    "n_estimators": [50, 100, 200, 400],
    "learning_rate": [0.01, 0.03, 0.1],
    "max_depth": [1, 2, 3, 4],
}

grid = GridSearchCV(
    GradientBoostingRegressor(random_state=42),
    param_grid=param_grid,
    scoring="neg_root_mean_squared_error",
    cv=5,
    n_jobs=-1,
)

grid.fit(X_train, y_train)

```

n_estimators сколько деревьев

learning_rate вклад каждого нового дерева

max_depth сложность базового дерева

Почему neg_RMSE?

GridSearchCV максимизирует score, поэтому sklearn использует отрицательное значение ошибки.

После выбора параметров настоящий RMSE считаем уже на test.

Реальный пример: редкий положительный класс

Высокая Accuracy может быть бесполезна — и поиск оптимизирует ровно ту метрику, которую мы ему дали.

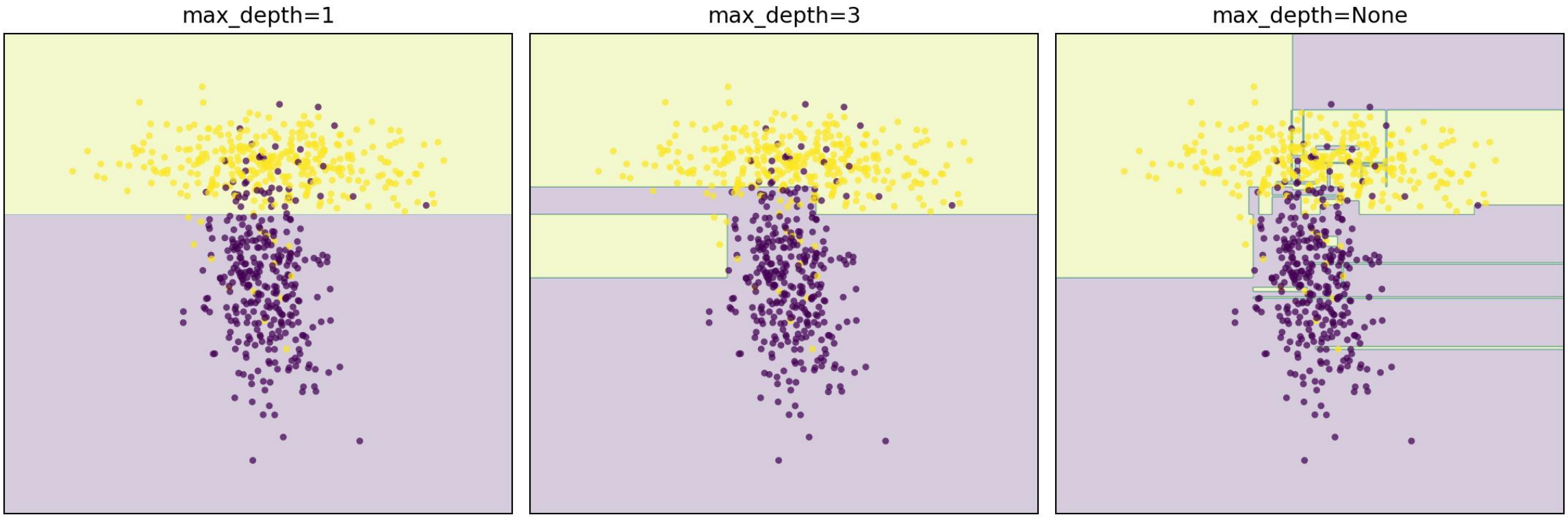


Настройка гиперпараметров — не магия. Если выбрать неправильный scoring, поиск очень хорошо оптимизирует неправильную цель.

То же относится к медицинскому скринингу: если опасно пропускать больных, Recall может быть важнее Accuracy.

Один гиперпараметр может полностью изменить модель

Пример: max_depth у дерева на одном и том же датасете.



UNDERFIT

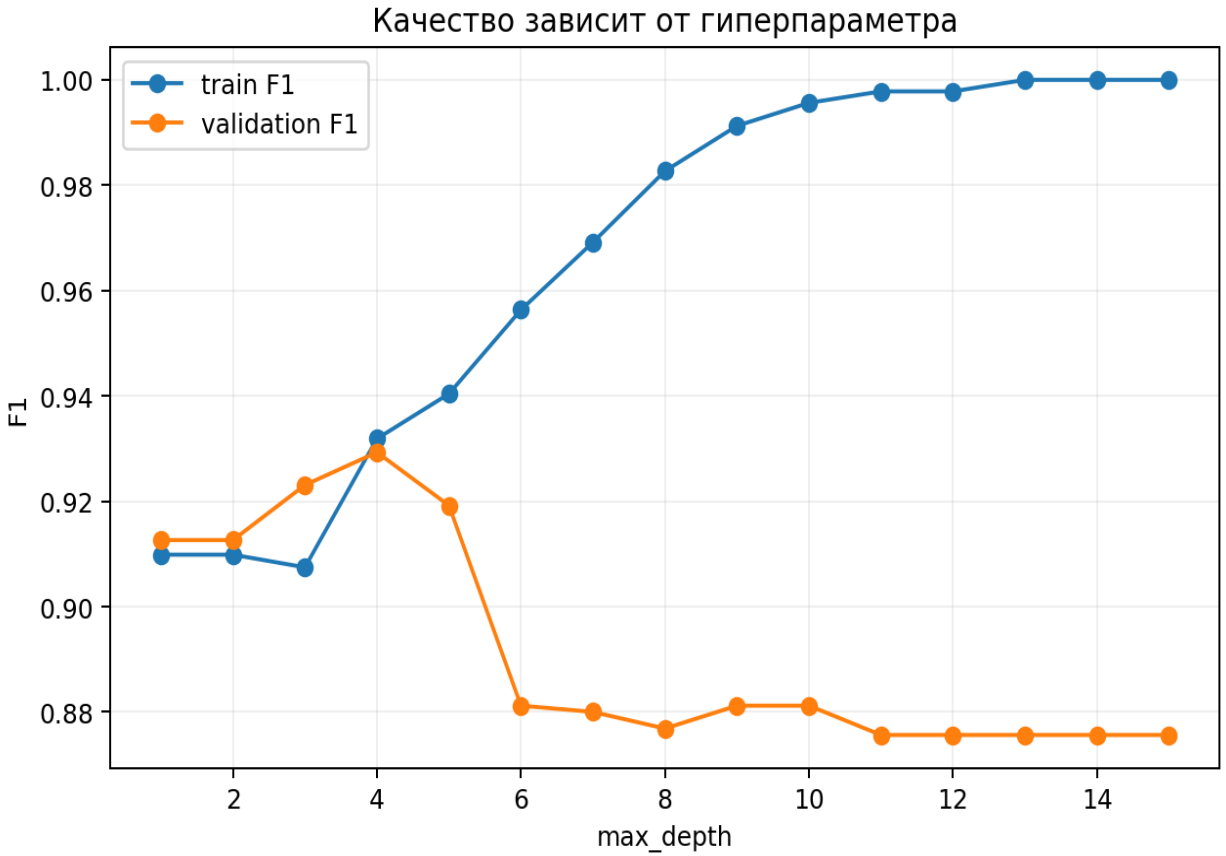
РАЗУМНАЯ СЛОЖНОСТЬ

OVERFIT

Смысл подбора - найти область, где модель достаточно сложная, но еще обобщает.

Ищем не максимум сложности, а максимум validation quality

Train обычно улучшается дольше, validation - нет.



Слева — модель слишком простая -> недообучение

Середина — лучший компромисс по CV

Справа — train высокий, а validation перестает расти или падает

Красивый код: импорты всегда сверху

PEP 8: сначала standard library, потом сторонние библиотеки, потом свои модули.

НЕ ТАК

```python
import pandas as pd

df = pd.read_csv("data.csv")

from sklearn.model_selection import GridSearchCV
...
import numpy as np
...
from sklearn.ensemble import RandomForestClassifier

```

ТАК

```python
import numpy as np
import pandas as pd

from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import GridSearchCV
from sklearn.pipeline import Pipeline

# ниже - загрузка данных и эксперимент

```

Не прячем import посреди ноутбука: человек должен сразу видеть зависимости.

Красивый код: убираем "магические числа"

Настройки эксперимента должны быть видны в одном месте.

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = RandomForestClassifier(random_state=42)
search = RandomizedSearchCV(..., n_iter=30, random_state=42)

```

```
RANDOM_STATE = 42
TEST_SIZE = 0.20
N_ITER = 30

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=TEST_SIZE, random_state=RANDOM_STATE
)

model = RandomForestClassifier(random_state=RANDOM_STATE)

```

Красивый код: одна функция вместо копипаста

Повторяющийся код почти всегда стоит вынести в функцию.

```python
def summarize_search(name, search):
    return {
        "search": name,
        "best_cv": search.best_score_,
        "best_params": search.best_params_,
    }

results = [
    summarize_search("grid", grid),
    summarize_search("random", random_search),
]

summary = pd.DataFrame(results)
```

Зачем?

- меньше копипаста
- одинаковая логика для всех экспериментов
- проще исправить ошибку в одном месте
- легче читать и проверять

Функция должна делать одну понятную вещь.

Pipeline + поиск = защита от утечки данных

Preprocessing должен переобучаться внутри каждого fold.

```python
pipe = Pipeline([
    ("prep", preprocess),
    ("model", RandomForestClassifier(
        random_state=RANDOM_STATE
    )),
])

param_grid = {
    "model__max_depth": [4, 8, None],
    "model__min_samples_leaf": [1, 2, 4],
}

```

Почему model__max_depth?

model	имя шага в Pipeline
__	разделитель
max_depth	параметр модели

Так GridSearchCV может настраивать объекты, вложенные внутрь Pipeline.

scoring: оптимизируем именно ту метрику, которая нужна

GridSearchCV всегда старается сделать score больше.

Классификация

scoring="f1"

чем больше F1 - тем лучше

Регрессия

scoring="neg_root_mean_squared_error"

возвращается -RMSE, потому что score должен максимизироваться

can you find  ?

